In [1]: # Reading and analyzing social data from the Christchurch hospital dataset
Python libraries: Pandas and Matplotlib/Seaborn/Scikit-learn/Statsmodels/Keras

In [2]: import os
os.getcwd()

Out[2]: 'C:\Users\Ubaldo\Desktop\Self_Learning\Hard_Dataset'

In [3]: import os
os.chdir('C:\Users\Ubaldo\Desktop\Self_Learning\Hard_Dataset'
os.getcwd()

In [4]: import pandas as pd
df = pd.read_csv('Hard_Dataset.csv')
df.head()

Out[4]:

	gender	age	ethnicity	smoking	hypertension	diabetes	stroke	heart_disease	alcohol_use	drug_use	hard_drinker
0	Male	50	Other	Yes	No	No	No	No	Yes	No	Yes
1	Male	50	Other	Yes	No	No	No	No	Yes	No	Yes
2	Male	50	Other	Yes	No	No	No	No	Yes	No	Yes
3	Male	50	Other	Yes	No	No	No	No	Yes	No	Yes
4	Male	50	Other	Yes	No	No	No	No	Yes	No	Yes

In [5]: df.info()

In [6]: df.describe()

Out[6]:

	age	alcohol_use	drug_use	hard_drinker
count	1.000000e+06	1.000000e+06	1.000000e+06	1.000000e+06
mean	4.000000e+01	1.000000e+00	1.000000e+00	1.000000e+00
std	1.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00
min	3.800000e+01	1.000000e+00	1.000000e+00	1.000000e+00
25%	3.900000e+01	1.000000e+00	1.000000e+00	1.000000e+00
50%	4.000000e+01	1.000000e+00	1.000000e+00	1.000000e+00
75%	4.100000e+01	1.000000e+00	1.000000e+00	1.000000e+00
max	4.200000e+01	1.000000e+00	1.000000e+00	1.000000e+00

In [7]: data_hard_count

In [8]: import matplotlib as mpl
%matplotlib inline
# Matplotlib's inline mode, so plots will be stored in the notebook.
# The %matplotlib command has a "magic" command because it's not a valid function.
# This is specific to IPython, and will only work inside an IPython notebook.
import matplotlib.pyplot as plt
plt.rcParams['figure.dpi'] = 100
plot size in pixels
%matplotlib inline

In [9]: data_hard_count.columns.tolist()

Out[9]: ['gender', 'age', 'ethnicity', 'smoking', 'hypertension', 'diabetes', 'stroke', 'heart_disease', 'alcohol_use', 'drug_use', 'hard_drinker']

In [10]: # Only keep the 5 and 6 stroke in the new column 'stroke' because these are male and 0 stroke for female.
# Drop the other rows.
df = df[df['stroke'] == 5 | df['stroke'] == 6]
df.head()

Out[10]:

	gender	age	ethnicity	smoking	hypertension	diabetes	stroke	heart_disease	alcohol_use	drug_use	hard_drinker
0	Male	50	Other	Yes	No	No	No	No	Yes	No	Yes
1	Male	50	Other	Yes	No	No	No	No	Yes	No	Yes
2	Male	50	Other	Yes	No	No	No	No	Yes	No	Yes
3	Male	50	Other	Yes	No	No	No	No	Yes	No	Yes
4	Male	50	Other	Yes	No	No	No	No	Yes	No	Yes

In [11]: df.shape

Out[11]: (1000, 10)

In [12]: # Drop data that is used to create the new column 'stroke'
df = df.drop(['stroke'], axis=1)
df.head()

Out[12]:

	gender	age	ethnicity	smoking	hypertension	diabetes	heart_disease	alcohol_use	drug_use	hard_drinker
0	Male	50	Other	Yes	No	No	No	Yes	No	Yes
1	Male	50	Other	Yes	No	No	No	Yes	No	Yes
2	Male	50	Other	Yes	No	No	No	Yes	No	Yes
3	Male	50	Other	Yes	No	No	No	Yes	No	Yes
4	Male	50	Other	Yes	No	No	No	Yes	No	Yes

In [13]: # Drop data that is used to create the new column 'stroke'
df = df.drop(['stroke'], axis=1)
df.shape

Out[13]: (1000, 10)

In [14]: # Check if there are any missing values
df.isnull().sum()

Out[14]: gender 0
age 0
ethnicity 0
smoking 0
hypertension 0
diabetes 0
heart_disease 0
alcohol_use 0
drug_use 0
hard_drinker 0
dtype: int64

In [15]: # Convert gender to 0 and 1
df['gender'] = df['gender'].replace({'Male': 1, 'Female': 0})
Convert smoking to 0 and 1
df['smoking'] = df['smoking'].replace({'Yes': 1, 'No': 0})
Convert hypertension to 0 and 1
df['hypertension'] = df['hypertension'].replace({'Yes': 1, 'No': 0})
Convert diabetes to 0 and 1
df['diabetes'] = df['diabetes'].replace({'Yes': 1, 'No': 0})
Convert heart_disease to 0 and 1
df['heart_disease'] = df['heart_disease'].replace({'Yes': 1, 'No': 0})
Convert alcohol_use to 0 and 1
df['alcohol_use'] = df['alcohol_use'].replace({'Yes': 1, 'No': 0})
Convert drug_use to 0 and 1
df['drug_use'] = df['drug_use'].replace({'Yes': 1, 'No': 0})
Convert hard_drinker to 0 and 1
df['hard_drinker'] = df['hard_drinker'].replace({'Yes': 1, 'No': 0})
df.head()

Out[15]:

	gender	age	ethnicity	smoking	hypertension	diabetes	heart_disease	alcohol_use	drug_use	hard_drinker
0	1	50	Other	1	0	0	0	1	0	1
1	1	50	Other	1	0	0	0	1	0	1
2	1	50	Other	1	0	0	0	1	0	1
3	1	50	Other	1	0	0	0	1	0	1
4	1	50	Other	1	0	0	0	1	0	1

In [16]: # Drop data that is used to create the new column 'stroke'
df = df.drop(['stroke'], axis=1)
df.shape

Out[16]: (1000, 10)

In [17]: # Check if there are any missing values
df.isnull().sum()

Out[17]: gender 0
age 0
ethnicity 0
smoking 0
hypertension 0
diabetes 0
heart_disease 0
alcohol_use 0
drug_use 0
hard_drinker 0
dtype: int64

In [18]: # Convert ethnicity to 0 and 1
df['ethnicity'] = df['ethnicity'].replace({'Other': 1, 'Caucasian': 0})
df.head()

Out[18]:

	gender	age	ethnicity	smoking	hypertension	diabetes	heart_disease	alcohol_use	drug_use	hard_drinker
0	1	50	1	1	0	0	0	1	0	1
1	1	50	1	1	0	0	0	1	0	1
2	1	50	1	1	0	0	0	1	0	1
3	1	50	1	1	0	0	0	1	0	1
4	1	50	1	1	0	0	0	1	0	1

In [19]: # Convert age to 0 and 1
df['age'] = df['age'].replace({'38': 0, '39': 0, '40': 1, '41': 1, '42': 1})
df.head()

Out[19]:

	gender	age	ethnicity	smoking	hypertension	diabetes	heart_disease	alcohol_use	drug_use	hard_drinker
0	1	1	1	1	0	0	0	1	0	1
1	1	1	1	1	0	0	0	1	0	1
2	1	1	1	1	0	0	0	1	0	1
3	1	1	1	1	0	0	0	1	0	1
4	1	1	1	1	0	0	0	1	0	1

In [20]: # Check if there are any missing values
df.isnull().sum()

Out[20]: gender 0
age 0
ethnicity 0
smoking 0
hypertension 0
diabetes 0
heart_disease 0
alcohol_use 0
drug_use 0
hard_drinker 0
dtype: int64

In [21]: # Convert data to numpy array
data = df.values
type(data), data.shape

Out[21]: (numpy.ndarray, (1000, 10))

In [22]: # Get the column names
cols = df.columns.tolist()
cols

Out[22]: ['gender', 'age', 'ethnicity', 'smoking', 'hypertension', 'diabetes', 'heart_disease', 'alcohol_use', 'drug_use', 'hard_drinker']

In [23]: # Unique values in the dataset
unique_values = df.nunique()
print(unique_values)

Out[23]: gender 2
age 2
ethnicity 2
smoking 2
hypertension 2
diabetes 2
heart_disease 2
alcohol_use 1
drug_use 1
hard_drinker 1
dtype: int64

In [24]: # Calculate the correlation matrix
corr = df.corr()
print(corr)

Out[24]: gender age ethnicity smoking hypertension \
gender 1.000000 NaN -0.032 0.032 0.032
age NaN NaN NaN NaN NaN
ethnicity -0.032 NaN 1.000 -0.032 -0.032
smoking 0.032 NaN -0.032 1.000 0.032
hypertension 0.032 NaN -0.032 0.032 1.000
diabetes 0.032 NaN -0.032 0.032 0.032
heart_disease 0.032 NaN -0.032 0.032 0.032
alcohol_use NaN NaN NaN NaN NaN
drug_use NaN NaN NaN NaN NaN
hard_drinker NaN NaN NaN NaN NaN

diabetes heart_disease alcohol_use drug_use hard_drinker
gender 0.032 0.032 NaN NaN NaN
age NaN NaN NaN NaN NaN
ethnicity -0.032 -0.032 NaN NaN NaN
smoking 0.032 0.032 NaN NaN NaN
hypertension 0.032 0.032 NaN NaN NaN
diabetes 1.000 0.032 NaN NaN NaN
heart_disease 0.032 1.000 NaN NaN NaN
alcohol_use NaN NaN NaN NaN NaN
drug_use NaN NaN NaN NaN NaN
hard_drinker NaN NaN NaN NaN NaN

In [25]: # Using Scikit-learnlibrary to find the correlation between the features and the target variable
# Import the Scikit-learn library
from sklearn.feature_selection import SelectKBest, f_classif
# Create a SelectKBest object with the f_classif function and k=10
k = 10
selector = SelectKBest(f_classif, k=k)
# Fit the selector to the data
selector.fit(df, df['hard_drinker'])
# Get the scores and p-values
scores = selector.scores_
pvalues = selector.pvalues_
# Print the scores and p-values
print('Scores:', scores)
print('P-values:', pvalues)

Out[25]: Scores: [1.000000 inf 1.000000 1.000000 1.000000
1.000000 1.000000 1.000000 1.000000 1.000000]
P-values: [0.31731051 0. 0.31731051 0.31731051 0.31731051
0.31731051 0.31731051 0.31731051 0.31731051 0.31731051]

In [26]: # Using Scikit-learnlibrary to find the correlation between the features and the target variable
# Import the Scikit-learn library
from sklearn.feature_selection import SelectKBest, f_classif
# Create a SelectKBest object with the f_classif function and k=10
k = 10
selector = SelectKBest(f_classif, k=k)
# Fit the selector to the data
selector.fit(df, df['hard_drinker'])
# Get the scores and p-values
scores = selector.scores_
pvalues = selector.pvalues_
# Print the scores and p-values
print('Scores:', scores)
print('P-values:', pvalues)

Out[26]: Scores: [1.000000 inf 1.000000 1.000000 1.000000
1.000000 1.000000 1.000000 1.000000 1.000000]
P-values: [0.31731051 0. 0.31731051 0.31731051 0.31731051
0.31731051 0.31731051 0.31731051 0.31731051 0.31731051]

In [27]: # Using Scikit-learnlibrary to find the correlation between the features and the target variable
# Import the Scikit-learn library
from sklearn.feature_selection import SelectKBest, f_classif
# Create a SelectKBest object with the f_classif function and k=10
k = 10
selector = SelectKBest(f_classif, k=k)
# Fit the selector to the data
selector.fit(df, df['hard_drinker'])
# Get the scores and p-values
scores = selector.scores_
pvalues = selector.pvalues_
# Print the scores and p-values
print('Scores:', scores)
print('P-values:', pvalues)

Out[27]: Scores: [1.000000 inf 1.000000 1.000000 1.000000
1.000000 1.000000 1.000000 1.000000 1.000000]
P-values: [0.31731051 0. 0.31731051 0.31731051 0.31731051
0.31731051 0.31731051 0.31731051 0.31731051 0.31731051]

In [28]: # Using Scikit-learnlibrary to find the correlation between the features and the target variable
# Import the Scikit-learn library
from sklearn.feature_selection import SelectKBest, f_classif
# Create a SelectKBest object with the f_classif function and k=10
k = 10
selector = SelectKBest(f_classif, k=k)
# Fit the selector to the data
selector.fit(df, df['hard_drinker'])
# Get the scores and p-values
scores = selector.scores_
pvalues = selector.pvalues_
# Print the scores and p-values
print('Scores:', scores)
print('P-values:', pvalues)

Out[28]: Scores: [1.000000 inf 1.000000 1.000000 1.000000
1.000000 1.000000 1.000000 1.000000 1.000000]
P-values: [0.31731051 0. 0.31731051 0.31731051 0.31731051
0.31731051 0.31731051 0.31731051 0.31731051 0.31731051]

In [29]: # Using Scikit-learnlibrary to find the correlation between the features and the target variable
# Import the Scikit-learn library
from sklearn.feature_selection import SelectKBest, f_classif
# Create a SelectKBest object with the f_classif function and k=10
k = 10
selector = SelectKBest(f_classif, k=k)
# Fit the selector to the data
selector.fit(df, df['hard_drinker'])
# Get the scores and p-values
scores = selector.scores_
pvalues = selector.pvalues_
# Print the scores and p-values
print('Scores:', scores)
print('P-values:', pvalues)

Out[29]: Scores: [1.000000 inf 1.000000 1.000000 1.000000
1.000000 1.000000 1.000000 1.000000 1.000000]
P-values: [0.31731051 0. 0.31731051 0.31731051 0.31731051
0.31731051 0.31731051 0.31731051 0.31731051 0.31731051]

In [30]: # Using Scikit-learnlibrary to find the correlation between the features and the target variable
# Import the Scikit-learn library
from sklearn.feature_selection import SelectKBest, f_classif
# Create a SelectKBest object with the f_classif function and k=10
k = 10
selector = SelectKBest(f_classif, k=k)
# Fit the selector to the data
selector.fit(df, df['hard_drinker'])
# Get the scores and p-values
scores = selector.scores_
pvalues = selector.pvalues_
# Print the scores and p-values
print('Scores:', scores)
print('P-values:', pvalues)

Out[30]: Scores: [1.000000 inf 1.000000 1.000000 1.000000
1.000000 1.000000 1.000000 1.000000 1.000000]
P-values: [0.31731051 0. 0.31731051 0.31731051 0.31731051
0.31731051 0.31731051 0.31731051 0.31731051 0.31731051]

In [31]: # Using Scikit-learnlibrary to find the correlation between the features and the target variable
# Import the Scikit-learn library
from sklearn.feature_selection import SelectKBest, f_classif
# Create a SelectKBest object with the f_classif function and k=10
k = 10
selector = SelectKBest(f_classif, k=k)
# Fit the selector to the data
selector.fit(df, df['hard_drinker'])
# Get the scores and p-values
scores = selector.scores_
pvalues = selector.pvalues_
# Print the scores and p-values
print('Scores:', scores)
print('P-values:', pvalues)

Out[31]: Scores: [1.000000 inf 1.000000 1.000000 1.000000
1.000000 1.000000 1.000000 1.000000 1.000000]
P-values: [0.31731051 0. 0.31731051 0.31731051 0.31731051
0.31731051 0.31731051 0.31731051 0.31731051 0.31731051]

In [32]: # Using Scikit-learnlibrary to find the correlation between the features and the target variable
# Import the Scikit-learn library
from sklearn.feature_selection import SelectKBest, f_classif
# Create a SelectKBest object with the f_classif function and k=10
k = 10
selector = SelectKBest(f_classif, k=k)
# Fit the selector to the data
selector.fit(df, df['hard_drinker'])
# Get the scores and p-values
scores = selector.scores_
pvalues = selector.pvalues_
# Print the scores and p-values
print('Scores:', scores)
print('P-values:', pvalues)

Out[32]: Scores: [1.000000 inf 1.000000 1.000000 1.000000
1.000000 1.000000 1.000000 1.000000 1.000000]
P-values: [0.31731051 0. 0.31731051 0.31731051 0.31731051
0.31731051 0.31731051 0.31731051 0.31731051 0.31731051]

In [33]: # Using Scikit-learnlibrary to find the correlation between the features and the target variable
# Import the Scikit-learn library
from sklearn.feature_selection import SelectKBest, f_classif
# Create a SelectKBest object with the f_classif function and k=10
k = 10
selector = SelectKBest(f_classif, k=k)
# Fit the selector to the data
selector.fit(df, df['hard_drinker'])
# Get the scores and p-values
scores = selector.scores_
pvalues = selector.pvalues_
# Print the scores and p-values
print('Scores:', scores)
print('P-values:', pvalues)

Out[33]: Scores: [1.000000 inf 1.000000 1.000000 1.000000
1.000000 1.000000 1.000000 1.000000 1.000000]
P-values: [0.31731051 0. 0.31731051 0.31731051 0.31731051
0.31731051 0.31731051 0.31731051 0.31731051 0.31731051]

In [34]: # Using Scikit-learnlibrary to find the correlation between the features and the target variable
# Import the Scikit-learn library
from sklearn.feature_selection import SelectKBest, f_classif
# Create a SelectKBest object with the f_classif function and k=10
k = 10
selector = SelectKBest(f_classif, k=k)
# Fit the selector to the data
selector.fit(df, df['hard_drinker'])
# Get the scores and p-values
scores = selector.scores_
pvalues = selector.pvalues_
# Print the scores and p-values
print('Scores:', scores)
print('P-values:', pvalues)

Out[34]: Scores: [1.000000 inf 1.000000 1.000000 1.000000
1.000000 1.000000 1.000000 1.000000 1.000000]
P-values: [0.31731051 0. 0.31731051 0.31731051 0.31731051
0.31731051 0.31731051 0.31731051 0.31731051 0.31731051]

In [35]: # Using Scikit-learnlibrary to find the correlation between the features and the target variable
# Import the Scikit-learn library
from sklearn.feature_selection import SelectKBest, f_classif
# Create a SelectKBest object with the f_classif function and k=10
k = 10
selector = SelectKBest(f_classif, k=k)
# Fit the selector to the data
selector.fit(df, df['hard_drinker'])
# Get the scores and p-values
scores = selector.scores_
pvalues = selector.pvalues_
# Print the scores and p-values
print('Scores:', scores)
print('P-values:', pvalues)

Out[35]: Scores: [1.000000 inf 1.000000 1.000000 1.000000
1.000000 1.000000 1.000000 1.000000 1.000000]
P-values: [0.31731051 0. 0.31731051 0.31731051 0.31731051
0.31731051 0.31731051 0.31731051 0.31731051 0.31731051]

In [36]: # Using Scikit-learnlibrary to find the correlation between the features and the target variable
# Import the Scikit-learn library
from sklearn.feature_selection import SelectKBest, f_classif
# Create a SelectKBest object with the f_classif function and k=10
k = 10
selector = SelectKBest(f_classif, k=k)
# Fit the selector to the data
selector.fit(df, df['hard_drinker'])
# Get the scores and p-values
scores = selector.scores_
pvalues = selector.pvalues_
# Print the scores and p-values
print('Scores:', scores)
print('P-values:', pvalues)

Out[36]: Scores: [1.000000 inf 1.000000 1.000000 1.000000
1.000000 1.000000 1.000000 1.000000 1.000000]
P-values: [0.31731051 0. 0.31731051 0.31731051 0.31731051
0.31731051 0.31731051 0.31731051 0.31731051 0.31731051]

In [37]: # Using Scikit-learnlibrary to find the correlation between the features and the target variable
# Import the Scikit-learn library
from sklearn.feature_selection import SelectKBest, f_classif
# Create a SelectKBest object with the f_classif function and k=10
k = 10
selector = SelectKBest(f_classif, k=k)
# Fit the selector to the data
selector.fit(df, df['hard_drinker'])
# Get the scores and p-values
scores = selector.scores_
pvalues = selector.pvalues_
# Print the scores and p-values
print('Scores:', scores)
print('P-values:', pvalues)

Out[37]: Scores: [1.000000 inf 1.000000 1.000000 1.000000
1.000000 1.000000 1.000000 1.000000 1.000000]
P-values: [0.31731051 0. 0.31731051 0.31731051 0.31731051
0.31731051 0.31731051 0.31731051 0.31731051 0.31731051]

In [38]: # Using Scikit-learnlibrary to find the correlation between the features and the target variable
# Import the Scikit-learn library
from sklearn.feature_selection import SelectKBest, f_classif
# Create a SelectKBest object with the f_classif function and k=10
k = 10
selector = SelectKBest(f_classif, k=k)
# Fit the selector to the data
selector.fit(df, df['hard_drinker'])
# Get the scores and p-values
scores = selector.scores_
pvalues = selector.pvalues_
# Print the scores and p-values
print('Scores:', scores)
print('P-values:', pvalues)

Out[38]: Scores: [1.000000 inf 1.000000 1.000000 1.000000
1.000000 1.000000 1.000000 1.000000 1.000000]
P-values: [0.31731051 0. 0.31731051 0.31731051 0.31731051
0.31731051 0.31731051 0.31731051 0.31731051 0.31731051]

In [39]: # Using Scikit-learnlibrary to find the correlation between the features and the target variable
# Import the Scikit-learn library
from sklearn.feature_selection import SelectKBest, f_classif
# Create a SelectKBest object with the f_classif function and k=10
k = 10
selector = SelectKBest(f_classif, k=k)
# Fit the selector to the data
selector.fit(df, df['hard_drinker'])
# Get the scores and p-values
scores = selector.scores_
pvalues = selector.pvalues_
# Print the scores and p-values
print('Scores:', scores)
print('P-values:', pvalues)

Out[39]: Scores: [1.000000 inf 1.000000 1.000000 1.000000
1.000000 1.000000 1.000000 1.000000 1.000000]
P-values: [0.31731051 0. 0.31731051 0.31731051 0.31731051
0.31731051 0.31731051 0.31731051 0.31731051 0.31731051]

In [40]: # Using Scikit-learnlibrary to find the correlation between the features and the target variable
# Import the Scikit-learn library
from sklearn.feature_selection import SelectKBest, f_classif
# Create a SelectKBest object with the f_classif function and k=10
k = 10
selector = SelectKBest(f_classif, k=k)
# Fit the selector to the data
selector.fit(df, df['hard_drinker'])
# Get the scores and p-values
scores = selector.scores_
pvalues = selector.pvalues_
# Print the scores and p-values
print('Scores:', scores)
print('P-values:', pvalues)

Out[40]: Scores: [1.000000 inf 1.000000 1.000000 1.000000
1.000000 1.000000 1.000000 1.000000 1.000000]
P-values: [0.31731051 0. 0.31731051 0.31731051 0.31731051
0.31731051 0.31731051 0.31731051 0.31731051 0.31731051]

In [41]: # Using Scikit-learnlibrary to find the correlation between the features and the target variable
# Import the Scikit-learn library
from sklearn.feature_selection import SelectKBest, f_classif
# Create a SelectKBest object with the f_classif function and k=10
k = 10
selector = SelectKBest(f_classif, k=k)
# Fit the selector to the data
selector.fit(df, df['hard_drinker'])
# Get the scores and p-values
scores = selector.scores_
pvalues = selector.pvalues_
# Print the scores and p-values
print('Scores:', scores)
print('P-values:', pvalues)

Out[41]: Scores: [1.000000 inf 1.000000 1.000000 1.000000
1.000000 1.000000 1.000000 1.000000 1.000000]
P-values: [0.31731051 0. 0.31731051 0.31731051 0.31731051
0.31731051 0.31731051 0.31731051 0.31731051 0.31731051]

In [42]: # Using Scikit-learnlibrary to find the correlation between the features and the target variable
# Import the Scikit-learn library
from sklearn.feature_selection import SelectKBest, f_classif
# Create a SelectKBest object with the f_classif function and k=10
k = 10
selector = SelectKBest(f_classif, k=k)
# Fit the selector to the data
selector.fit(df, df['hard_drinker'])
# Get the scores and p-values
scores = selector.scores_
pvalues = selector.pvalues_
# Print the scores and p-values
print('Scores:', scores)
print('P-values:', pvalues)

Out[42]: Scores: [1.000000 inf 1.000000 1.000000 1.000000
1.000000 1.000000 1.000000 1.000000 1.000000]
P-values: [0.31731051 0. 0.31731051 0.31731051 0.31731051
0.31731051 0.31731051 0.31731051 0.31731051 0.31731051]

In [43]: # Using Scikit-learnlibrary to find the correlation between the features and the target variable
# Import the Scikit-learn library
from sklearn.feature_selection import SelectKBest, f_classif
# Create a SelectKBest object with the f_classif function and k=10
k = 10
selector = SelectKBest(f_classif, k=k)
# Fit the selector to the data
selector.fit(df, df['hard_drinker'])
# Get the scores and p-values
scores = selector.scores_
pvalues = selector.pvalues_
# Print the scores and p-values
print('Scores:', scores)
print('P-values:', pvalues)

Out[43]: Scores: [1.000000 inf 1.000000 1.000000 1.000000
1.000000 1.000000 1.000000 1.000000 1.000000]
P-values: [0.31731051 0. 0.31731051 0.31731051 0.31731051
0.31731051 0.31731051 0.31731051 0.31731051 0.31731051]

In [44]: # Using Scikit-learnlibrary to find the correlation between the features and the target variable
# Import the Scikit-learn library
from sklearn.feature_selection import SelectKBest, f_classif
# Create a SelectKBest object with the f_classif function and k=10
k = 10
selector = SelectKBest(f_classif, k=k)
# Fit the selector to the data
selector.fit(df, df['hard_drinker'])
# Get the scores and p-values
scores = selector.scores_
pvalues = selector.pvalues_
# Print the scores and p-values
print('Scores:', scores)
print('P-values:', pvalues)

Out[44]: Scores: [1.000000 inf 1.000000 1.000000 1.000000
1.000000 1.000000 1.000000 1.000000 1.000000]
P-values: [0.31731051 0. 0.31731051 0.31731051 0.31731051
0.31731051 0.31731051 0.31731051 0.31731051 0.31731051]

In [45]: # Using Scikit-learnlibrary to find the correlation between the features and the target variable
# Import the Scikit-learn library
from sklearn.feature_selection import SelectKBest, f_classif
# Create a SelectKBest object with the f_classif function and k=10
k = 10
selector = SelectKBest(f_classif, k=k)
# Fit the selector to the data
selector.fit(df, df['hard_drinker'])
# Get the scores and p-values
scores = selector.scores_
pvalues = selector.pvalues_
# Print the scores and p-values
print('Scores:', scores)
print('P-values:', pvalues)

Out[45]: Scores: [1.000000 inf 1.000000 1.000000 1.000000
1.000000 1.000000 1.000000 1.000000 1.000000]
P-values: [0.31731051 0. 0.31731051 0.31731051 0.31731051
0.31731051 0.31731051 0.31731051 0.31731051 0.31731051]

In [46]: # Using Scikit-learnlibrary to find the correlation between the features and the target variable
# Import the Scikit-learn library
from sklearn.feature_selection import SelectKBest, f_classif
# Create a SelectKBest object with the f_classif function and k=10
k = 10
selector = SelectKBest(f_classif, k=k)
# Fit the selector to the data
selector.fit(df, df['hard_drinker'])
# Get the scores and p-values
scores = selector.scores_
pvalues = selector.pvalues_
# Print the scores and p-values
print('Scores:', scores)
print('P-values:', pvalues)

Out[46]: Scores: [1.000000 inf 1.000000 1.000000 1.000000
1.000000 1.000000 1.000000 1.000000 1.000000]
P-values: [0.31731051 0. 0.31731051 0.31731051 0.31731051
0.31731051 0.31731051 0.31731051 0.31731051 0.31731051]

In [47]: # Using Scikit-learnlibrary to find the correlation between the features and the target variable
# Import the Scikit-learn library
from sklearn.feature_selection import SelectKBest, f_classif
# Create a SelectKBest object with the f_classif function and k=10
k = 10
selector = SelectKBest(f_classif, k=k)
# Fit the selector to the data
selector.fit(df, df['hard_drinker'])
# Get the scores and p-values
scores = selector.scores_
pvalues = selector.pvalues_
# Print the scores and p-values
print('Scores:', scores)
print('P-values:', pvalues)

Out[47]: Scores: [1.000000 inf 1.000000 1.000000 1.000000
1.000000 1.000000 1.000000 1.000000 1.000000]
P-values: [0.31731051 0. 0.31731051 0.31731051 0.31731051
0.31731051 0.31731051 0.31731051 0.31731051 0.31731051]

In [48]: # Using Scikit-learnlibrary to find the correlation between the features and the target variable
# Import the Scikit-learn library
from sklearn.feature_selection import SelectKBest, f_classif
# Create a SelectKBest object with the f_classif function and k=10
k = 10
selector = SelectKBest(f_classif, k=k)
# Fit the selector to the data
selector.fit(df, df['hard_drinker'])
# Get the scores and p-values
scores = selector.scores_
pvalues = selector.pvalues_
# Print the scores and p-values
print('Scores:', scores)
print('P-values:', pvalues)

Out[48]: Scores: [1.000000 inf 1.000000 1.000000 1.000000
1.000000 1.000000 1.000000 1.000000 1.000000]
P-values: [0.31731051 0. 0.31731051 0.31731051 0.31731051
0.31731051 0.31731051 0.31731051 0.31731051 0.31731051]

In [49]: # Using Scikit-learnlibrary to find the correlation between the features and the target variable
# Import the Scikit-learn library
from sklearn.feature_selection import SelectKBest, f_classif
# Create a SelectKBest object with the f_classif function and k=10
k = 10
selector = SelectKBest(f_classif, k=k)
# Fit the selector to the data
selector.fit(df, df['hard_drinker'])
# Get the scores and p-values
scores = selector.scores_
pvalues = selector.pvalues_
# Print the scores and p-values
print('Scores:', scores)
print('P-values:', pvalues)

Out[49]: Scores: [1.000000 inf 1.000000 1.000000 1.000000
1.000000 1.000000 1.000000 1.000000 1.000000]
P-values: [0.31731051 0. 0.31731051 0.31731051 0.31731051
0.31731051 0.31731051 0.31731051 0.31731051 0.31731051]

In [50]: # Using Scikit-learnlibrary to find the correlation between the features and the target variable
# Import the Scikit-learn library
from sklearn.feature_selection import SelectKBest, f_classif
# Create a SelectKBest object with the f_classif function and k=10
k = 10
selector = SelectKBest(f_classif, k=k)
# Fit the selector to the data
selector.fit(df, df['hard_drinker'])
# Get the scores and p-values
scores = selector.scores_
pvalues = selector.pvalues_
# Print the scores and p-values
print('Scores:', scores)
print('P-values:', pvalues)

Out[50]: Scores: [1.000000 inf 1.000000 1.000000 1.000000
1.000000 1.000000 1.000000 1.000000 1.000000]
P-values: [0.31731051 0. 0.31731051 0.31731051 0.31731051
0.31731051 0.31731051 0.31731051 0.31731051 0.31731051]

In [51]: # Using Scikit-learnlibrary to find the correlation between the features and the target variable
# Import the Scikit-learn library
from sklearn.feature_selection import SelectKBest, f_classif
# Create a SelectKBest object with the f_classif function and k=10
k = 10
selector = SelectKBest(f_classif, k=k)
# Fit the selector to the data
selector.fit(df, df['hard_drinker'])
# Get the scores and p-values
scores = selector.scores_
pvalues = selector.pvalues_
# Print the scores and p-values
print('Scores:', scores)
print('P-values:', pvalues)

Out[51]: Scores: [1.000000 inf 1.000000 1.000000 1.000000
1.000000 1.000000 1.000000 1.000000 1.000000]
P-values: [0.31731051 0. 0.31731051 0.31731051 0.31731051
0.31731051 0.31731051 0.31731051 0.31731051 0.31731051]

In [52]: # Using Scikit-learnlibrary to find the correlation between the features and the target variable
# Import the Scikit-learn library
from sklearn.feature_selection import SelectKBest, f_classif
# Create a SelectKBest object with the f_classif function and k=10
k = 10
selector = SelectKBest(f_classif, k=k)
# Fit the selector to the data
selector.fit(df, df['hard_drinker'])
# Get the scores and p-values
scores = selector.scores_
pvalues = selector.pvalues_
# Print the scores and p-values
print('Scores:', scores)
print('P-values:', pvalues)

Out[52]: Scores: [1.000000 inf 1.000000 1.000000 1.000000
1.000000 1.000000 1.000000 1.000000 1.000000]
P-values: [0.31731051 0. 0.31731051 0.31731051 0.31731051
0.31731051 0.31731051 0.31731051 0.31731051 0.31731051]

In [53]: # Using Scikit-learnlibrary to find the correlation between the features and the target variable
# Import the Scikit-learn library
from sklearn.feature_selection import SelectKBest, f_classif
# Create a SelectKBest object with the f_classif function and k=10
k = 10
selector = SelectKBest(f_classif, k=k)
# Fit the selector to the data
selector.fit(df, df['hard_drinker'])
# Get the scores and p-values
scores = selector.scores_
pvalues = selector.pvalues_
# Print the scores and p-values
print('Scores:', scores)
print('P-values:', pvalues)

Out[53]: Scores: [1.000000 inf 1.000000 1.000000 1.000000
1.000000 1.000000 1.000000 1.000000 1.000000]
P-values: [0.31731051 0. 0.31731051 0.31731051 0.31731051
0.31731051 0.31731051 0.31731051 0.31731051 0.31731051]

In [54]: # Using Scikit-learnlibrary to find the correlation between the features and the target variable
# Import the Scikit-learn library
from sklearn.feature_selection import SelectKBest, f_classif
# Create a SelectKBest object with the f_classif function and k=10
k = 10
selector = SelectKBest(f_classif, k=k)
# Fit the selector to the data
selector.fit(df, df['hard_drinker'])
# Get the scores and p-values
scores = selector.scores_
pvalues = selector.pvalues_
# Print the scores and p-values
print('Scores:', scores)
print('P-values:', pvalues)

Out[54]: Scores: [1.000000 inf 1.000000 1.000000 1.000000
1.000000 1.000000 1.000000 1.000000 1.000000]
P-values: [0.31731051 0. 0.31731051 0.31731051 0.31731051
0.31731051 0.31731051 0.31731051 0.31731051 0.31731051]

In [55]: # Using Scikit-learnlibrary to find the correlation between the features and the target variable
# Import the Scikit-learn library
from sklearn.feature_selection import SelectKBest, f_classif
# Create a SelectKBest object with the f_classif function and k=10
k = 10
selector = SelectKBest(f_classif, k=k)
# Fit the selector to the data
selector.fit(df, df['hard_drinker'])
# Get the scores and p-values
scores = selector.scores_
pvalues = selector.pvalues_
# Print the scores and p-values
print('Scores:', scores)
print('P-values:', pvalues)

Out[55]: Scores: [1.000000 inf 1.000000 1.000000 1.000000
1.000000 1.000000 1.000000 1.000000 1.000000]
P-values: [0.31731051 0. 0.31731051 0.31731051 0.31731051
0.31731051 0.31731051 0.31731051 0.31731051 0.31731051]

In [56]: # Using Scikit-learnlibrary to find the correlation between the features and the target variable
# Import the Scikit-learn library
from sklearn.feature_selection import SelectKBest, f_classif
# Create a SelectKBest object with the f_classif function and k=10
k = 10
selector = SelectKBest(f_classif, k=k)
# Fit the selector to the data
selector.fit(df, df['hard_drinker'])
# Get the scores and p-values
scores = selector.scores_
pvalues = selector.pvalues_
# Print the scores and p-values
print('Scores:', scores)
print('P-values:', pvalues)

Out[56]: Scores: [1.000000 inf 1.000000 1.000000 1.000000
1.000000 1.000000 1.000000 1.000000 1.000000]
P-values: [0.31731051 0. 0.31731051 0.31731051 0.31731051
0.31731051 0.31731051 0.31731051 0.31731051 0.31731051]

In [57]: # Using Scikit-learnlibrary to find the correlation between the features and the target variable
# Import the Scikit-learn library
from sklearn.feature_selection import SelectKBest, f_classif
# Create a SelectKBest object with the f_classif function and k=10
k = 10
selector = SelectKBest(f_classif, k=k)
# Fit the selector to the data
selector.fit(df, df['hard_drinker'])
# Get the scores and p-values
scores = selector.scores_
pvalues = selector.pvalues_
# Print the scores and p-values
print('Scores:', scores)
print('P-values:', pvalues)

Out[57]: Scores: [1.000000 inf 1.000000 1.000000 1.000000
1.000000 1.000000 1.000000 1.000000 1.000000]
P-values: [0.31731051 0. 0.31